

Lecture 15: Conjugate Gradient Methods

Instructor Proof Notes

Color Legend

Green: Step numbering and substeps **Orange:** Key concepts **Purple:** Main results **Blue:** Teaching notes

Proof Roadmap

Theorem	Time	Board-Friendly	Priority
Theorem 1: Convergence in n Steps	3–5 min	Yes	Full
Theorem 2: CG Produces Q-Conjugate Directions	8–10 min	Sketch	Key Result
Theorem 3: Equivalence of β_k Formulas	5–8 min	Yes	Full
Theorem 4: Convergence Rate Bound for CG	8–10 min	Sketch	Key Result

1 Theorem 1: Convergence in n Steps for CDM

Statement: Let Q be **symmetric positive definite (SPD)**. In the **Conjugate Direction Method (CDM)**, starting from x_1 with Q-conjugate directions $d_1, d_2, \dots, d_n$, the iterate x_{n+1} is the **global minimizer** of $f(x) = \frac{1}{2}x^T Qx + c^T x + \text{const}$ in **at most n steps**.

Proof:

Step 1: Recall that by definition, in CDM we have $x_{k+1} = x_k + \lambda_k d_k$ where λ_k is chosen by exact line search, and $d_1, \dots, d_n$ are Q-conjugate.

Step 2: By **Theorem 1 from Lecture 12**, Q-conjugate directions are **linearly independent**. Thus, $\{d_1, \dots, d_n\}$ spans $\mathbb{R}^n$.

Step 3: Write the displacement from x_1 as

$$x_{k+1} = x_1 + \sum_{i=1}^k \lambda_i d_i = x_1 + D^k \lambda^k,$$

where $D^k = [d_1 \cdots d_k]$ and $\lambda^k = [\lambda_1, \dots, \lambda_k]^T$.

Step 4: By **Theorem 3 from Lecture 12** (Expanding Subspace Theorem), x_{k+1} minimizes f over the affine subspace $\{x_1\} + \text{span}(d_1, \dots, d_k)$.

Step 5: When $k = n$, we have $\text{span}(d_1, \dots, d_n) = \mathbb{R}^n$ (since the n linearly independent vectors span the entire space). Thus, the affine subspace becomes all of $\mathbb{R}^n$.

Step 6: Therefore, x_{n+1} minimizes f over all of $\mathbb{R}^n$.

Step 7: Since Q is SPD, f is strictly convex, so the global minimum is unique and satisfies $\nabla f(x_{n+1}) = Qx_{n+1} + c = 0$.

CDM converges to the global minimizer in at most n iterations.

Teaching Notes: Theorem 1

Board Strategy: This is the payoff theorem that motivates everything. Keep it snappy (3–5 min); it's a corollary of the Lecture 12 results.

Key Insight: Unlike gradient descent (infinitely many steps), CDM is *finite-terminating*: exactly n steps for an n -dimensional problem. This is the main advantage of conjugate direction methods.

Intuition: With each iteration, we expand the subspace over which we've minimized. After n iterations covering the whole space, we're done.

Caveat: This finite termination assumes exact arithmetic and exact line search. In practice, round-off errors accumulate.

Timing: 3–5 minutes. Can be delivered quickly after referencing Lecture 12.

Common Student Confusion

Q: “If CDM converges in n steps, why do we need CG? Isn't CDM already optimal?”

A: Good question! CDM requires us to choose the conjugate directions $d_1, \dots, d_n$ in advance. But we don't know them beforehand—computing them requires knowledge of the Hessian Q and solving additional systems. CG is brilliant because it *generates* the conjugate directions automatically during the iterations using only gradient information and one previous direction.

2 Theorem 2: CG Produces Q-Conjugate Directions

Statement: The **Conjugate Gradient (CG) method** is defined by the recurrence:

$$d_1 = -\nabla_1, \tag{1}$$

$$d_{k+1} = -\nabla_{k+1} + \beta_k d_k, \tag{2}$$

where $\beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}$ and $x_{k+1} = x_k + \lambda_k d_k$ with λ_k chosen by exact line search. Then the directions $d_1, d_2, \dots$ are **Q-conjugate**, meaning $d_i^T Q d_j = 0$ for all $i \neq j$.

Proof Sketch:

Step 1: We prove by induction that $d_k^T Q d_i = 0$ for all $i < k$.

Base case ($k = 2$):

Step 2: We have $d_2 = -\nabla_2 + \beta_1 d_1$ where $\beta_1 = \frac{\nabla_2^T Q d_1}{d_1^T Q d_1}$.

Step 3: Compute $d_2^T Q d_1$:

$$d_2^T Q d_1 = (-\nabla_2 + \beta_1 d_1)^T Q d_1 = -\nabla_2^T Q d_1 + \beta_1 (d_1^T Q d_1).$$

Step 4: Substitute the definition of β_1 :

$$d_2^T Q d_1 = -\nabla_2^T Q d_1 + \frac{\nabla_2^T Q d_1}{d_1^T Q d_1} \cdot (d_1^T Q d_1) = -\nabla_2^T Q d_1 + \nabla_2^T Q d_1 = 0. \quad \checkmark$$

Inductive step:

Step 5: Assume $d_i^T Q d_j = 0$ for all $1 \leq j < i < k$. We must show $d_{k+1}^T Q d_i = 0$ for all $i \leq k$.

Step 6: For $i = k$: By construction of $\beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}$, the same cancellation as in the base case gives $d_{k+1}^T Q d_k = 0$.

Step 7: For $i < k$: Write

$$d_{k+1}^T Q d_i = (-\nabla_{k+1} + \beta_k d_k)^T Q d_i = -\nabla_{k+1}^T Q d_i + \beta_k (d_k^T Q d_i).$$

Step 8: By the induction hypothesis, $d_k^T Q d_i = 0$ (since $i < k$). So the second term vanishes. For the first term, use **Theorem 2 from Lecture 12**: $\nabla_{k+1}^T d_i = 0$. By differentiating the quadratic model and using the line search condition, one can show $\nabla_{k+1}^T Q d_i = 0$ for $i < k$ as well. *[Detailed verification requires properties of quadratic functions; see Nocedal & Wright.]*

Step 9: Therefore, $d_{k+1}^T Q d_i = 0 + 0 = 0$.

By induction, all CG directions are Q-conjugate.

Teaching Notes: Theorem 2

Board Strategy: The complete rigorous proof of the inductive step is technical. **Sketch the base case fully** (showing the cancellation), state the inductive assumption clearly, and sketch the two-term decomposition in Step 7. State Step 8 as a consequence of Lecture 12 theory.

Key Insight: The *magic* of CG is the formula for β_k . By choosing $\beta_k = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}$, we ensure $d_{k+1} \perp_Q d_k$ using only the previous direction. This is why CG has linear memory complexity!

Comparison to CDM: In CDM, we must specify all n directions upfront. In CG, we generate them iteratively—and remarkably, the one-parameter update rule guarantees Q-conjugacy with *all* previous directions (not just the immediately preceding one), thanks to the structure of quadratic functions.

Diagram Idea: Draw 2D example showing $\nabla_1, d_1 = -\nabla_1$, then $\nabla_2, d_2 = -\nabla_2 + \beta_1 d_1$. Show how $\beta_1 d_1$ “bends” $-\nabla_2$ to be perpendicular (in the Q -metric) to d_1 .

Timing: 8–10 minutes. Full proof of Step 8 can be assigned as reading.

Common Student Confusion

Q: “If we only ensure Q-conjugacy with d_k , how do we ensure Q-conjugacy with $d_1, \dots, d_{k-1}$?”

A: Excellent question! This is one of the most beautiful facts in optimization. By a nontrivial argument (Step 8), using the special structure of quadratics and the line search, Q-conjugacy with all previous directions is *automatically guaranteed* if we maintain Q-conjugacy with the immediately previous direction. The induction proof makes this precise. This is why CG is so efficient—we only store d_k , not the entire history.

3 Theorem 3: Equivalence of β_k Formulas for Quadratics

Statement: In CG applied to the quadratic $f(x) = \frac{1}{2}x^T Qx + c^T x$ with exact line search, the following two formulas for β_k are **equivalent**:

$$\beta_k^{(\text{original})} = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}, \quad (3)$$

$$\beta_k^{(\text{FR})} = \frac{\|\nabla_{k+1}\|^2}{\|\nabla_k\|^2}. \quad (4)$$

Proof:

Step 1: By **Theorem 2 from Lecture 12**, under exact line search with Q-conjugate directions, we have $\nabla_{k+1}^T d_i = 0$ for all $i = 1, \dots, k$.

Step 2: In particular, $\nabla_{k+1}^T d_k = 0$, which means

$$\nabla_{k+1}^T (-\nabla_k + \beta_{k-1} d_{k-1}) = 0.$$

Step 3: Expand:

$$-\nabla_{k+1}^T \nabla_k + \beta_{k-1} \nabla_{k+1}^T d_{k-1} = 0.$$

Step 4: By Theorem 2 from Lecture 12, $\nabla_{k+1}^T d_{k-1} = 0$. Thus,

$$\nabla_{k+1}^T \nabla_k = 0.$$

[The gradient is orthogonal (in the standard Euclidean sense, not Q-sense) to all previous search directions.]

Step 5: Now, for the equivalence: Start with the original formula

$$\beta_k^{(\text{original})} = \frac{\nabla_{k+1}^T Q d_k}{d_k^T Q d_k}.$$

Step 6: From the CG update, $d_k = -\nabla_k + \beta_{k-1} d_{k-1}$. Compute the numerator:

$$\nabla_{k+1}^T Q d_k = \nabla_{k+1}^T Q (-\nabla_k + \beta_{k-1} d_{k-1}).$$

Step 7: Expand:

$$\nabla_{k+1}^T Q d_k = -\nabla_{k+1}^T Q \nabla_k + \beta_{k-1} \nabla_{k+1}^T Q d_{k-1}.$$

Step 8: By the structure of quadratic functions and the line search optimality condition at step k , we have $\nabla_{k+1}^T Q d_{k-1} = 0$ *[(this uses the fact that $\nabla_{k+1}^T d_{k-1} = 0$ from Step 4, and properties of Q)]*. Thus,

$$\nabla_{k+1}^T Q d_k = -\nabla_{k+1}^T Q \nabla_k.$$

Step 9: For the denominator, similarly,

$$d_k^T Q d_k = (-\nabla_k + \beta_{k-1} d_{k-1})^T Q (-\nabla_k + \beta_{k-1} d_{k-1}).$$

Step 10: Expanding and using $\nabla_k^T Q d_{k-1} = 0$ (line search optimality from the previous step):

$$d_k^T Q d_k = \nabla_k^T Q \nabla_k.$$

Step 11: Therefore,

$$\beta_k^{(\text{original})} = \frac{-\nabla_{k+1}^T Q \nabla_k}{\nabla_k^T Q \nabla_k}.$$

Step 12: By the definition of $\nabla_{k+1} = Q x_{k+1} + c = Q(x_k + \lambda_k d_k) + c = \nabla_k + \lambda_k Q d_k$, we have

$$\nabla_{k+1} - \nabla_k = \lambda_k Q d_k,$$

$$\text{so } Q d_k = \frac{\nabla_{k+1} - \nabla_k}{\lambda_k}.$$

Step 13: Insert into Step 11:

$$\beta_k^{(\text{original})} = \frac{-\nabla_{k+1}^T \frac{\nabla_{k+1} - \nabla_k}{\lambda_k}}{\nabla_k^T \frac{\nabla_{k+1} - \nabla_k}{\lambda_k}} = \frac{-\nabla_{k+1}^T (\nabla_{k+1} - \nabla_k)}{\nabla_k^T (\nabla_{k+1} - \nabla_k)}.$$

Step 14: By Step 4, $\nabla_{k+1}^T \nabla_k = 0$. Thus,

$$-\nabla_{k+1}^T (\nabla_{k+1} - \nabla_k) = -\nabla_{k+1}^T \nabla_{k+1} + \nabla_{k+1}^T \nabla_k = -\|\nabla_{k+1}\|^2.$$

Step 15: Also, using orthogonality:

$$\nabla_k^T (\nabla_{k+1} - \nabla_k) = \nabla_k^T \nabla_{k+1} - \nabla_k^T \nabla_k = 0 - \|\nabla_k\|^2 = -\|\nabla_k\|^2.$$

Step 16: Therefore,

$$\beta_k^{(\text{original})} = \frac{-\|\nabla_{k+1}\|^2}{-\|\nabla_k\|^2} = \frac{\|\nabla_{k+1}\|^2}{\|\nabla_k\|^2} = \beta_k^{(\text{FR})}.$$

The two formulas for β_k are equivalent for quadratics with exact line search.

Teaching Notes: Theorem 3

Board Strategy: This is a computational result showing that two formulas are equivalent. On the board, highlight the key ingredient: **gradient orthogonality** (Step 4). Then sketch the algebraic manipulations, jumping over some details.

Key Insight: The **Fletcher-Reeves (FR)** formula $\beta_k^{(\text{FR})} = \frac{\|\nabla_{k+1}\|^2}{\|\nabla_k\|^2}$ uses only gradient norms, not the matrix Q itself. This is huge for practical implementation—we don't need to compute Qd_k separately!

Gradient Orthogonality: The fact that $\nabla_{k+1}^T \nabla_k = 0$ (in the standard dot product) is a consequence of the line search and Q -conjugacy. It's remarkable and worth emphasizing.

Practical Implication: In implementations, we use $\beta_k^{(\text{FR})}$ because it's cheaper: just norms of gradients. The original formula $\beta_k^{(\text{original})}$ requires an explicit matrix-vector product Qd_k .

Timing: 5–8 minutes. The proof is mostly algebra; focus on the structure.

Common Student Confusion

Q: “If the FR formula doesn't use Q , how does it ensure Q -conjugacy?”

A: Great question! The FR formula is equivalent to the original formula, so it automatically ensures Q -conjugacy. However, this equivalence holds *exactly* only for quadratics with exact line search. For nonquadratic functions or inexact line search, the FR formula may break down; other formulas like Polak-Ribière are sometimes more robust.

4 Theorem 4: Convergence Rate Bound for CG

Statement: In the quadratic case with CG applied to $f(x) = \frac{1}{2}x^T Qx + c^T x$, after k iterations, the error in the Q -norm satisfies

$$\|x_k - x^*\|_Q \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|x_0 - x^*\|_Q,$$

where $\kappa = \lambda_{\max}(Q)/\lambda_{\min}(Q)$ is the **condition number** of Q , and $\|v\|_Q = \sqrt{v^T Q v}$.

Proof Sketch:

Step 1: The error $e_k = x_k - x^*$ satisfies the recurrence $e_{k+1} = e_k + \lambda_k d_k$.

Step 2: CG can be analyzed via **Chebyshev polynomials**. The key observation is that the iterate x_k minimizes f over the expanding Krylov subspace

$$K_k = \text{span}\{\nabla_1, Q\nabla_1, Q^2\nabla_1, \dots, Q^{k-1}\nabla_1\}.$$

Step 3: Define the error in the Q -norm: $\|e_k\|_Q^2 = e_k^T Q e_k$. This measures the energy norm of the error.

Step 4: By minimizing f over the Krylov subspace K_k , CG solves the problem of minimizing $\|e_k\|_Q^2$ over all x_k of the form $x_k = x_0 + y$ where $y \in K_k$.

Step 5: The optimal approximation of x^* from K_k can be characterized using **Chebyshev polynomials**. Specifically, if P_k is the degree- k Chebyshev polynomial (scaled and shifted to $[0, 1]$), then

$$\|e_k\|_Q \leq 2\|P_k\left(\frac{Q}{\lambda_{\max}}\right)\| \|e_0\|_Q.$$

Step 6: *[The precise statement involves the Chebyshev polynomial of the first kind T_k and properties of eigenvalue distributions. We omit the full technical details here.]*

Step 7: The bound on the Chebyshev polynomial growth is

$$\max_{t \in [0, 1]} |P_k(t)| \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k,$$

where the polynomial is optimized to have the smallest maximum on the normalized eigenvalue interval $[1/\kappa, 1]$.

Step 8: Combining Steps 5 and 7 yields the bound

$$\|e_k\|_Q \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k \|e_0\|_Q.$$

CG convergence rate is bounded by a factor depending on $\sqrt{\kappa}$ rather than κ (as in steepest descent)

Teaching Notes: Theorem 4

Board Strategy: This is a deep result; sketch the main ideas rather than deriving everything. Focus on: (1) The Krylov subspace connection, (2) Chebyshev polynomials as the optimal tool, (3) The scaling by $\sqrt{\kappa}$ vs. κ .

Key Insight: CG's convergence rate depends on $\sqrt{\kappa}$, whereas steepest descent's rate depends on κ . This is a dramatic improvement. For example, if $\kappa = 100$:

- Steepest descent: rate $(1 - 1/\kappa)^k = (0.99)^k$ requires ≈ 460 steps for 10^{-2} accuracy.
- CG: rate $((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^k = (0.818)^k$ requires ≈ 25 steps.

Chebyshev Polynomials: These are the optimal polynomial approximations in the minimax sense. CG essentially solves the problem “find the best polynomial approximation to $1/t$ on the spectrum of Q ”, which is a classical application of Chebyshev polynomials.

Krylov Subspaces: Modern iterative methods (CG, GMRES, etc.) work in Krylov subspaces. This perspective unifies many algorithms.

Reference: See Saad, *Iterative Methods for Sparse Linear Systems*, Chapter 6, for a complete treatment.

Timing: 8–10 minutes for sketching the ideas. Full proofs should be assigned as reading or supplementary material.

Common Student Confusion

Q: “Why is the bound $((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^k$ better than steepest descent’s $(1 - 1/\kappa)^k$?”

A: Let’s compare exponents. Define $r_{\text{CG}} = (\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ and $r_{\text{SD}} = 1 - 1/\kappa$. For large κ :

$$r_{\text{CG}} \approx 1 - \frac{2}{\sqrt{\kappa}} \quad \text{and} \quad r_{\text{SD}} \approx 1 - \frac{1}{\kappa}.$$

Since $\sqrt{\kappa} \ll \kappa$ for large κ , CG’s rate is much better. For $\kappa = 100$: $r_{\text{CG}} \approx 0.818$ vs. $r_{\text{SD}} \approx 0.99$, a tenfold improvement in convergence rate.

Q: “The bound looks like CG converges in $O(\sqrt{\kappa} \log(1/\epsilon))$ steps. Is that tight?”

A: Yes, and this is often achieved in practice. However, the bound is in the worst case (worst conditioning of Q). Superlinear convergence can occur if eigenvalues cluster; CG can identify clusters and converge faster.

Summary and Connections

The Big Picture of Lecture 15:

1. **Theorem 1** shows CDM converges in at most n steps—finite termination is the dream.
2. **Theorem 2** shows that CG *generates* Q-conjugate directions automatically using a one-parameter update rule.
3. **Theorem 3** shows that the practical formula for β_k (using gradient norms) is equivalent to the original formula—making CG computationally efficient.
4. **Theorem 4** quantifies the convergence rate: $\sqrt{\kappa}$ dependence vastly beats the κ dependence of steepest descent.

Why This Matters: CG bridges theory and practice. Theoretically, it’s a finite-terminating method for quadratics. Practically, it’s an efficient iterative method for large sparse systems, with superb scaling in the condition number.

Practical Implementation Notes:

- **Finite termination in theory:** In exact arithmetic, x_n is optimal after n steps. In floating-point arithmetic, round-off errors accumulate; we usually stop when $\|\nabla_k\| < \epsilon$.
- **Krylov subspace view:** CG iteratively builds the Krylov subspace $K_k = \text{span}\{\nabla_1, Q\nabla_1, \dots, Q^{k-1}\nabla_1\}$ and minimizes f over the affine shift of this subspace.
- **Matrix-free implementations:** The algorithm only requires matrix-vector products Qd_k ; we never explicitly form or store Q (crucial for large-scale problems).
- **Preconditioning:** For poorly conditioned systems (κ large), preconditioned CG (PCG) scales the problem: solve $M^{-1}Qx = M^{-1}c$ for a preconditioner $M \approx Q$.
- **Non-quadratic extension:** CG can be extended to general nonconvex functions (e.g., Polak-Ribière formula). It's no longer finite-terminating, but often converges faster than steepest descent.

Recommended Homework / Discussion

- Implement CG from scratch for a small $n \times n$ SPD system and verify convergence in at most n steps.
- Compare steepest descent and CG on a 2D or 3D quadratic with different condition numbers. Plot $\|x_k - x^*\|$ vs. iteration k and verify the theoretical rate.
- Derive the Polak-Ribière formula $\beta_k^{\text{PR}} = \frac{\nabla_{k+1}^T(\nabla_{k+1} - \nabla_k)}{d_k^T(\nabla_{k+1} - \nabla_k)}$ and discuss why it's more robust for non-quadratics.
- Explore preconditioning: implement PCG with a simple preconditioner (e.g., diagonal scaling or incomplete Cholesky) and observe the condition number reduction.
- Read about GMRES (generalized minimal residual) as a non-symmetric extension of CG; discuss the added complexity of maintaining orthogonality.

Connections to Lecture 14 and Beyond:

- **From Lecture 14:** Line search and exact minimization along a direction are used here for the step size λ_k .
- **Versus steepest descent:** Both use gradient information, but CG uses conjugate directions instead of gradient directions, achieving $\sqrt{\kappa}$ scaling vs. κ for steepest descent.
- **Toward Lecture 16:** Newton and quasi-Newton methods (BFGS, L-BFGS) approximate the Hessian and achieve even better rates. CG can be viewed as an implicit second-order method.
- **Krylov subspace methods:** CG is the simplest member of a family (GMRES, BiCG, etc.) for solving linear systems and nonlinear optimization.